

DOCKER

setup guide: <https://www.youtube.com/watch?v=5Mfe54238xE>

Concept : https://www.youtube.com/watch?v=H8Lyj2D_cWo

1. The Problem: "It works on my machine!"

Imagine you build a brilliant little Python app on your laptop. It works perfectly. You send the code to your friend, or you try to put it on a live web server, and suddenly it crashes. Why?

Maybe your friend has a different version of Python installed. Maybe they are missing a background library your app needs. Maybe you are on a Mac and the server is running Linux.

In the old days, fixing this meant writing massive instruction manuals on how to set up the computer *exactly* right before running your code. It was a nightmare.

2. The Solution: The Shipping Container Analogy

Before the 1950s, global shipping was chaotic. People loaded sacks of coffee, wooden crates of apples, and steel car parts individually onto ships. Every ship had to be loaded differently. Then, the industry invented the **standardized shipping container**. Suddenly, it didn't matter what was inside—apples or car parts—the container was always the exact same size, and the cranes, ships, and trucks all knew exactly how to handle it.

Docker does this for software. Docker lets you pack up your application, along with *everything* it needs to run (the exact Python version, the specific libraries, the settings), into a standardized digital box.

Because it is in this standardized box, you can drop it onto a Mac, a Windows PC, or a giant Google Cloud server, and **it will run exactly the same way every single time.**

3. The Core Vocabulary

To use Docker, you only really need to understand three main concepts to start:

- **Dockerfile:** The Blueprint. This is a simple text file you write with instructions. It says things like, *"Start with a basic Linux computer, install Python 3.9, copy my code into it, and press play."*
- **Docker Image:** The Snapshot. When you run your Dockerfile, Docker builds an "Image." Think of this as a frozen-in-time, read-only template of your app. It's the physical shipping container packed and ready to go.
- **Docker Container:** The Live App. When you take an Image and actually turn it on, it becomes a Container. You can run one container, or you can run 100 identical containers from the exact same Image.

4. Why not just use a Virtual Machine?

You might be wondering, "Isn't this just a tiny computer inside my computer?" Sort of, but it's much more efficient.

Traditional Virtual Machines (VMs) are heavy. If you want to run three different apps in VMs, you have to install an entire operating system (like Windows or Linux) three separate times. Docker containers, on the other hand, *share* the operating system of the computer they are running on. This makes them incredibly lightweight, meaning they start up in milliseconds instead of minutes.

Docker vs. Virtual Machines (VMs)

- **Virtual Machines:** These virtualize the entire operating system, making them heavier and slower to start.
- **Docker:** This virtualizes only the application layer by sharing the host OS kernel, which is why it is significantly faster and more lightweight.

Syntax

Here is your starter guide to Docker syntax.

Part 1: The **Dockerfile** Syntax (The Blueprint)

The **Dockerfile** is just a text file (no file extension) where you write a top-to-bottom recipe for your environment. Docker reads these instructions in order.

Here are the 5 most important keywords:

Dockerfile

```
# 1. FROM: Always the first line. It sets your starting point (base image).
# Instead of a full OS, we use a "slim" version to keep the image size tiny and efficient.
FROM python:3.10-slim

# 2. WORKDIR: Creates a folder inside the container and moves you into it.
# Everything that happens next will happen inside this /app directory.
WORKDIR /app

# 3. COPY: Copies files from your local computer into the container.
# The syntax is: COPY [local_source] [container_destination]
COPY requirements.txt .

# 4. RUN: Executes terminal commands inside the container during the build phase.
# Usually used for installing dependencies.
RUN pip install --no-cache-dir -r requirements.txt

# (Copying the rest of the application code over)
COPY . .

# 5. CMD: The final command that runs when the container actually starts.
# Unlike RUN (which happens during the build), CMD is the main process keeping the app alive.
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Part 2: The Core Terminal Commands (The Action)

Once you have your code and that **Dockerfile** sitting in the same folder, you use your terminal to bring it to life.

1. Building the Image

You need to tell Docker to read the `Dockerfile` and bake the image.

Bash

```
docker build -t my-python-api .
```

- `build`: The command to create an image.
- `t my-python-api`: "Tags" (names) your image so you can find it later.
- `.`: Tells Docker the `Dockerfile` is in your current directory. Don't forget the dot!

2. Running the Container

Now that the image is built, let's turn it on.

Bash

```
docker run -d -p 8000:8000 my-python-api
```

- `run`: Starts the container from an image.
- `d`: "Detached" mode. It runs in the background so you can keep using your terminal.
- `p 8000:8000`: "Port mapping". It connects port 8000 on your actual computer to port 8000 inside the isolated container. (Crucial for web apps and APIs).

3. Checking What's Running

To see all containers currently alive and running on your machine:

Bash

```
docker ps
```

(This will output a table showing the Container ID, the image it's running, and the ports).

4. Stopping a Container

Grab the Container ID from the `docker ps` command and stop it:

Bash

```
docker stop <container_id>
```

The Golden Rule of Docker

Every time a container restarts, any files generated or changed inside it are erased. It starts completely fresh from the original Image. This ensures absolute reliability, but it means you have to map external storage if you want to save persistent data (like logs or database records).

1. How to Shut It Down Safely

Since your terminal is currently "attached" to the running containers (showing you the live logs), shutting it down is easy:

- **The Quick Stop:** Just click into that terminal and press `Ctrl + C`. Docker will safely send a shutdown signal to Uvicorn and Streamlit and power down the containers.
- **The Clean Sweep:** If you want to stop the containers *and* remove the invisible network tying them together (good for saving RAM when playing video games or doing other heavy tasks), type this in

the terminal:Bash

```
docker-compose down
```

2. How to Turn It Back On Tomorrow

When you wake up tomorrow and want to show this to a friend, you do not need to wait 10 minutes for it to build again.

Open VS Code, open a terminal in your root folder, and simply run:

Bash

```
docker-compose up
```

(Notice we dropped the `--build` flag. Docker will use the cached blueprints and boot the whole system in about 3 seconds!)

3. How to Update Your Code

If you decide to change the color of the Streamlit canvas, add a new title, or change the FastAPI logic, your workflow is:

1. Stop the containers (`Ctrl + C`).
2. Edit and save your Python files.
3. Run `docker-compose up --build` . Docker will instantly snap the new code into the containers and restart.

4. What about Docker Desktop?

You can mostly ignore it! The only time you really need to open the Docker Desktop app is for "spring cleaning." If you build 20 different projects over the next year, those 1GB images will stack up on your hard drive. You can go into the **Images** tab in the app and delete old ones to free up space.
